



TM416

# Programming mapp Axis

**Requirements**

|                  |                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Training modules | TM240 – Ladder Diagram (LD)<br>or<br>TM246 – Structured Text (ST)<br>TM415 – Working with integrated motion control |
| Software         | Automation Studio 4.9<br>Automation Runtime 4.91<br>mapp Motion Technology Package                                  |
| Hardware         | X20 controller + ACOPOS product family<br>or<br>Simulation                                                          |

# Table of contents

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| 1 Introduction.....                                                                  | 4  |
| 1.1 Learning objectives.....                                                         | 4  |
| 1.2 PLCopen standard.....                                                            | 4  |
| 2 mapp Technology.....                                                               | 6  |
| 2.1 Instructions for using mapp Technology components.....                           | 6  |
| 2.2 Diagnostic options for mapp Technology components.....                           | 6  |
| 2.3 mapp Motion libraries.....                                                       | 8  |
| 3 Integrating an axis in the control project.....                                    | 9  |
| 3.1 The MpAxis library.....                                                          | 9  |
| 3.2 Creating a program and adding MpAxisBasic.....                                   | 9  |
| 3.3 Function block operation and status evaluation.....                              | 12 |
| 3.4 Overview of drive states.....                                                    | 13 |
| 4 Programming tips.....                                                              | 16 |
| 4.1 Uses of control structures.....                                                  | 16 |
| 5 Error indicators in the application program.....                                   | 17 |
| 6 Additional MpAxisBasic functions.....                                              | 19 |
| 7 The PLCopen Motion library (McAxis).....                                           | 20 |
| 7.1 Compatibility of McAxis and MpAxis.....                                          | 20 |
| 8 Automation.....                                                                    | 23 |
| 8.1 Automating an axis application.....                                              | 23 |
| 9 Summary.....                                                                       | 25 |
| 10 Solutions.....                                                                    | 27 |
| 10.1 Solution: Switch on the controller automatically and perform direct homing..... | 27 |

# 1 Introduction

The application program is a fundamental part of each positioning task. This is where signals from the process are evaluated and movement commands are configured and transferred to the drive. The drives in the process are usually controlled by an automatic sequence from the application program. The diagnostics for the drives also includes the application software and the HMI application. The user must be familiar with the available tools in order to create a draft of the positioning application.

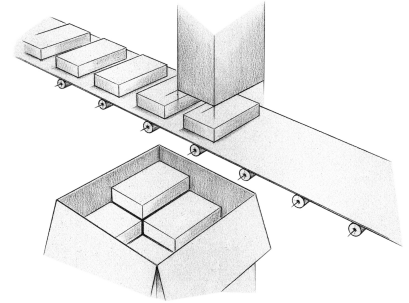


Figure 1: Schematic illustration of a palletizer

This training module concentrates on programming mapp Motion components using mapp Motion libraries and integrating them into the application program.

Working through different exercises will help us to understand the behavior of function blocks and provide us with important basic knowledge for applying these functions. Ladder Diagram is used in the image source files and examples for calling function blocks. Any programming language can be used for creating the application program.

## 1.1 Learning objectives

- Participants will learn where to find necessary information in Automation Help.
- Participants will get an overview of the mapp Technology concept for single-axis applications.
- Participants will be able to integrate function blocks in their control program and will get an overview of the structure of library McAxis.
- Participants will know about the necessary steps for drive preparation and performing base movements.
- Participants will know the typical structure of an application program and understand the relationships in the PLCopen state diagram.
- Participants will be able to implement error handling in their application themselves.

## 1.2 PLCopen standard

The role played by software in automation systems is continually increasing. This is associated with increased complexity for applications, which in turn results in additional work for development, support and maintenance of the software.

The PLCopen organization was created with the goal of unifying and standardizing different areas of activity in the field of automation. The areas of activity include drive technology, safety technology, the IEC 61131-3 standard as well as OPC UA and XML.

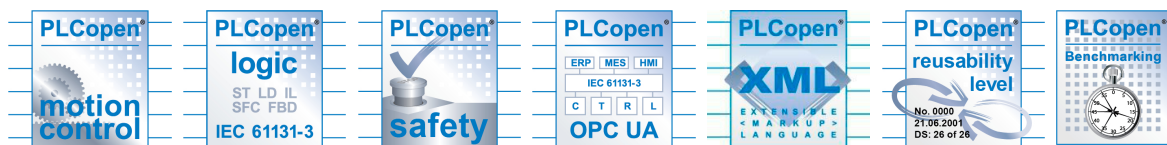


Table 1: Different fields of activity of the PLCopen organization

Detailed information about the PLCopen organization and its activities can be found on its website: <http://www.plcopen.org/>

Guidelines are being developed to ensure that different system solutions are all operated in the same way. Suppliers of automation solutions who are members of this organization provide uniform software interfaces that are defined via PLCopen. B&R is an active member of the PLCopen organization.

**Fully supported by B&R**

PLCopen-compliant function blocks are available for programming the B&R drive solution. With mapp Motion, project creation is done quickly and easily using standardized function blocks.

## 2 mapp Technology

With mapp Technology<sup>1</sup>, we offer users an easy-to-use interface for implementing comprehensive functionality. Many complex operations, such as loading and saving recipe data, controlling a drive axis and recording process values, can be implemented quickly and easily using mapp Technology components.



Figure 2: mapp Technology logo

mapp Technology unites configuration and programming. The functionality itself is implemented in the application program using standard libraries. In addition, mapp provides configuration interfaces. Similar to hardware modules, these are used to configure the functionality of the mapp components without programming.

User access to mapp Technology is enabled via independent Technology Packages. They are loaded via the B&R website or the Upgrades dialog box in Automation Studio.



mapp Technology  
Motion control \ mapp Motion  
Project management \ Workspace \ Upgrades

### 2.1 Instructions for using mapp Technology components



The MpLink from the Configuration View is transferred to the function block in the program with the ADR() function. This establishes the connection from the configuration to the programming code of the mapp function blocks.

#### Configuration files

There is a configuration available for every mapp component. The configuration is created and modified in the Configuration View in Automation Studio or the application program. For additional information about the mapp configuration, see Automation Help.

#### Transfer settings



When using mapp Motion, it is recommended to use the option "Keep PV values" in the transfer settings.



mapp Technology \ Concept \ mapp components

- mapp Link
- Configuration instead of programming
- Application interface
- Application interface \ Function blocks – "Download behavior" section

Motion control \ mapp Motion \ Configuration

### 2.2 Diagnostic options for mapp Technology components

mapp Technology components can be monitored and diagnosed via several different methods. The following is a list of the diagnostic options in Automation Studio, in web-based diagnostics and in the HMI application.

<sup>1</sup> mapp Technology stands for "Modular APplication technology".

## Programming languages in monitor mode

In many cases, the first access is monitor mode during application software programming. The values of the process variables are visible directly in the context of the program code. All function blocks used to operate mapp Technology components have outputs "Error" and "StatusID" that can be used to perform initial diagnostics.

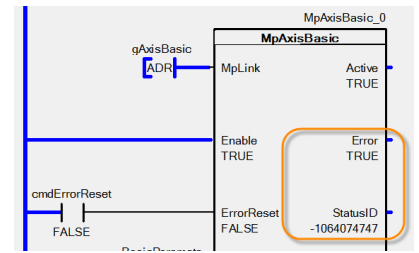


Figure 3: Ladder Diagram program in monitor mode

## Watch window

The Watch window is opened in the Logical View using the program shortcut menu or in the software configuration by selecting **<Watch>**. The instance variables of the function blocks used are added using the toolbar or shortcut menu.

The information structure and the outputs Error, StatusID and Command-Busy can be used to diagnose the current state, for example. A description of these parameters and the error numbers can be found in the description of the respective function block.

| Name             | Value                    |
|------------------|--------------------------|
| Active           | TRUE                     |
| Error            | TRUE                     |
| StatusID         | -1064074747              |
| CommandBusy      | FALSE                    |
| CommandAborted   | FALSE                    |
| PowerOn          | FALSE                    |
| IsHomed          | TRUE                     |
| Info             |                          |
| AxisInitialized  | TRUE                     |
| ReadyToPowerOn   | TRUE                     |
| PLCOpenState     | mpAXIS_DISABLED          |
| Diag             |                          |
| StatusID         |                          |
| ID               | mpAXIS_ERR_PLC_OPEN      |
| Severity         | mpCOM_SEV_ERROR          |
| Code             | 33285                    |
| Internal         |                          |
| ID               | -1073712530              |
| Severity         | mpCOM_SEV_ERROR          |
| Facility         | mpCOM_FAC_ARCORE         |
| Code             | 29294                    |
| ExecutingCommand | mpAXIS_CMD_MOVE_VELOCITY |

Figure 4: Instance variable of the MpAxisBasic function block in the Watch window

## Logger

In the case of an error, additional information from the mapp Technology component is added into the logger file with the name "Motion". The error number can be searched for directly in Automation Help or called by pressing the **<F1 key>**. Additional information is provided in the details section of the highlighted Logger entry. If there is a PLCOpen error, for example, the affected function and the cause of the error are described clearly in the details section of the Logger entry.

If there are a lot of entries, the hierarchy and filter can also be used to find specific information. Additional programming is not required to filter out mapp Logger entries.

| Severity    | Time                       | ID          | Area | Entered by | Origin                                                      | Description | ASCII Data          | Binary Data |
|-------------|----------------------------|-------------|------|------------|-------------------------------------------------------------|-------------|---------------------|-------------|
| Error       | 2019-06-28 14:16:46.984000 | -1067317248 | B&R  | gAxis_1    | Command: MoveAdditive failed                                |             | 45 02 01 62 00      |             |
| Error       | 2019-06-28 14:16:46.984000 | -1067317245 | B&R  | gAxis_1    | The FB cannot be used in the current state (AxisDisabled)   |             | 45 00 01 00 00      |             |
| Information | 2019-06-28 14:16:46.979000 | 1080213511  | B&R  | gAxis_1    | Command: MoveAdditive                                       |             | 45 02 01 62 00      |             |
| Information | 2019-06-28 14:16:46.979000 | 1083181809  | B&R  | gAxis_1    | MpAxisBasic.MoveAdditive call                               |             | 21 2D 00 21 4D 6F   |             |
| Information | 2019-06-28 14:10:39.424000 | 1080164356  | B&R  | gAxis_2    | The axis state has changed from AxisStartup to AxisDisabled |             | 45 00 01 00 00 4... |             |
| Information | 2019-06-28 14:10:39.424000 | 1080164356  | B&R  | gAxis_1    | The axis state has changed from AxisStartup to AxisDisabled |             | 45 00 01 00 00 4... |             |
| Information | 2019-06-28 14:10:39.414000 | 1080214353  | B&R  | gAxis_2    | Command: AxisInitialization finished                        |             | 45 FF FF 64 00      |             |
| Information | 2019-06-28 14:10:39.414000 | 1080214353  | B&R  | gAxis_1    | Command: AxisInitialization finished                        |             | 45 FF FF 64 00      |             |
| Information | 2019-06-28 14:10:39.504000 | 1080214353  | B&R  | gAxis_2    | Command: ChannelInitialization finished                     |             | 45 FF FF 64 00      |             |
| Information | 2019-06-28 14:10:39.504000 | 1080214353  | B&R  | gAxis_1    | Command: ChannelInitialization finished                     |             | 45 FF FF 64 00      |             |
| Information | 2019-06-28 14:10:38.384000 | 1080213525  | B&R  | gAxis_2    | Command: Start initialization of channel                    |             |                     |             |
| Information | 2019-06-28 14:10:38.384000 | 1080213525  | B&R  | gAxis_1    | Command: Start initialization of channel                    |             |                     |             |
| Information | 2019-06-28 14:10:38.374000 | 1080213524  | B&R  | gAxis_2    | Command: Start initialization of axis                       |             |                     |             |
| Information | 2019-06-28 14:10:38.374000 | 1080213524  | B&R  | gAxis_1    | Command: Start initialization of axis                       |             |                     |             |
| Information | 2019-06-28 14:10:37.779000 | 1083181809  | B&R  | gAxis_2    | MpAxisBasic.Enable call                                     |             | 21 2D 00 21 45 6... |             |
| Information | 2019-06-28 14:10:37.779000 | 1083181809  | B&R  | gAxis_1    | MpAxisBasic.Enable call                                     |             | 21 2D 00 21 45 6... |             |

Figure 5: Function block error in the Logger window

## System Diagnostics Manager

The Logger file for the mapp components in SDM can be saved in System Diagnostics Manager. The SDM can be embedded directly into the mapp View or Visual Components HMI application using the HTML control.

### Alarm management with MpAlarmX

mapp AlarmX collects and manages both mapp alarms and user-specific alarms. The alarms are configured using Automation Studio, managed in the application and then displayed in an HMI application or exported as a file. The entries displayed can be filtered by group, time, priority, etc.

#### Additional information about mapp Services components, e.g. mapp AlarmX:

- TM270 – Configuration, commissioning and diagnosis of mapp Services



Diagnostics and service \ Diagnostics tools \

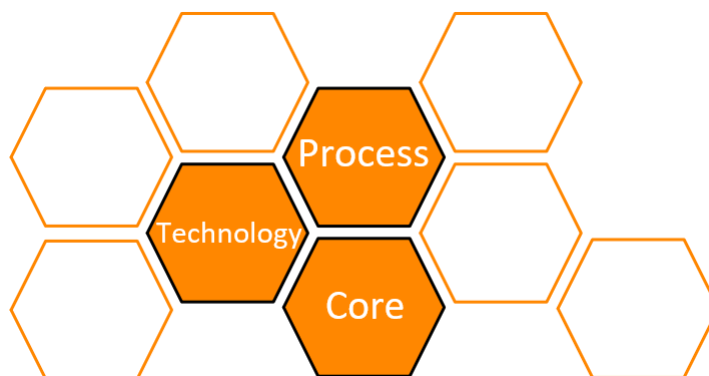
- Logger
- Watch window
- Monitor mode
- System Diagnostics Manager (SDM)
- mapp Cockpit

Services \ mapp Services \ MpAlarmX: Alarm management

Visualization \ mapp View \ Widgets \ Media \ WebViewer

Visualization \ Visual Components VC4 Manual \ Control reference \ HTML view

## 2.3 mapp Motion libraries



Within mapp Motion, the individual libraries are split up into 3 areas.

The underlying concept is listed in the help section in the order of necessary function blocks as follows.

- **Process libraries:** These are higher-level function sequences that operate multiple axes or axis groups.
- **Technology libraries:** These are simple blocks that are particularly helpful during commissioning. The range of functions is usually a sum of blocks from the core libraries.
- **Core libraries:** These are blocks with one functionality each as well as device-specific function blocks that require certain hardware and are very hardware-related (e.g. ACOPOS servo family).



Motion control \ mapp Motion \ Programming \ Application program \ Libraries

## 3 Integrating an axis in the control project

The procedure for creating a drive configuration in Automation Studio has been explained in the training module "TM415 – Working with integrated motion control". Drive movements could already be performed using mapp Cockpit. The following example shows how an application program for controlling axis movements is created step by step. An axis reference for accessing the axis component will be needed first. The axis reference has already been generated by inserting the drive configuration.

Automation Help shows how to prepare the drive with MpAxisbasic and how to execute a base movement.



Motion control \ mapp Motion \ Getting started

### 3.1 The MpAxis library

The MpAxis mapp Technology library offers standard functions for controlling and configuring drive axes. The function block MpAxisBasic is used for drive control. The following function categories for single-axis control are covered by function block MpAxisBasic:

- Drive preparation (switching on, homing)
- Base movements (absolute, additive, continuous)
- Error handling (acknowledgment)



The MpAxis library is based on the function blocks in library McAxis. Both libraries are therefore compatible with one another and can be implemented together in applications, see 7.1 "Compatibility of McAxis and MpAxis" on page 20.

All mapp components are based on open standards, technology functions and libraries that can be directly used by the user in the application.

Details regarding the used function blocks and functionality can be taken from the corresponding section in Automation Help.



Figure 6: mapp components are based on open standards, technology functions and libraries



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Technology \ MpAxis \  
Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAxis

### 3.2 Creating a program and adding MpAxisBasic

Now it is necessary to expand the Automation Studio project by calling function block MpAxisBasic. The MpAxis library is added to the Logical View.

Function block MpAxisBasic is added to a program and connected to the axis reference. The procedure can be found in the "Getting started" section of Automation Help.

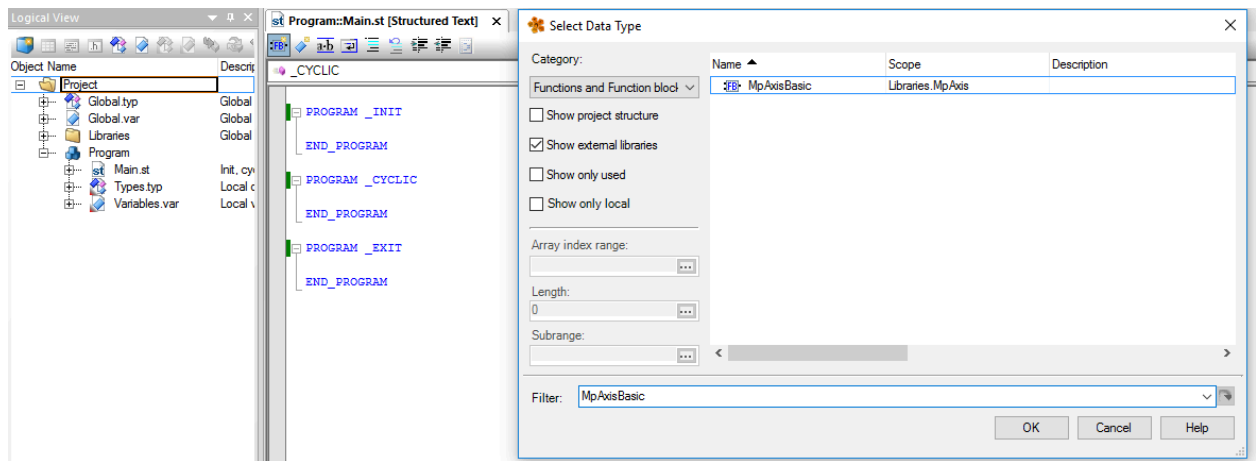


Figure 7: Add MpAxisBasic.

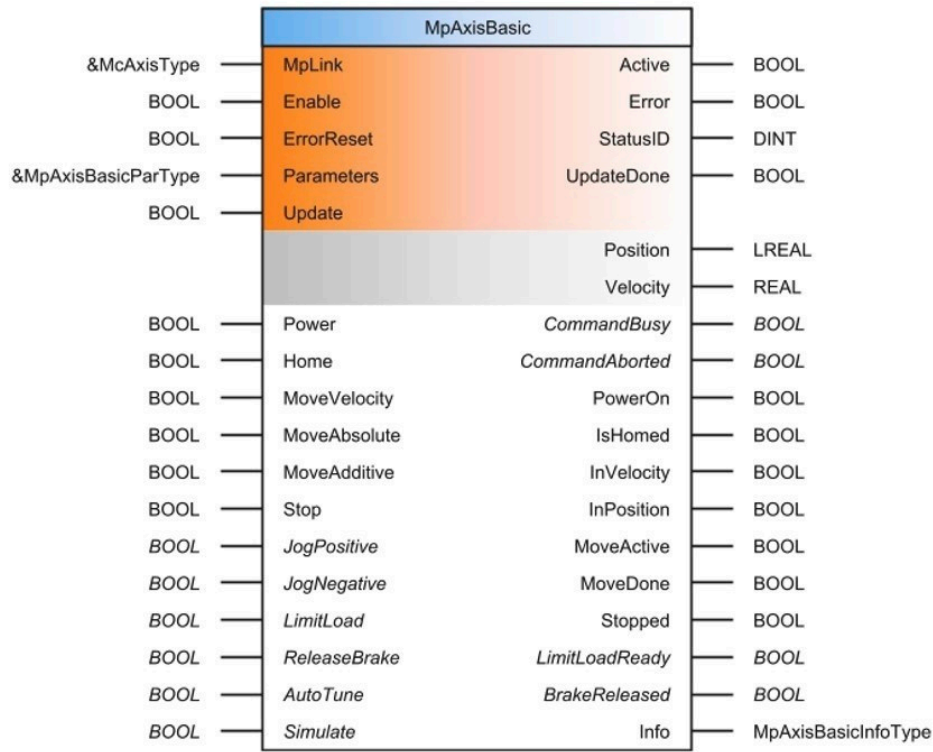


Figure 8: MpAxisBasic covers all basic functionalities of an axis.

### 3.2.1 Connect the axis reference and the movement parameters

It is necessary to transfer the axis reference and the movement parameters so that an axis can be accessed and basic movements can be performed.

#### Using the axis reference

Adding the drive configuration automatically created a variable for use in the function blocks. This allows the connection from the axis component to the actual drive hardware to be established on the POWERLINK network. The address of the axis reference is specified for all function blocks using parameter "MpLink". In Ladder Diagram, either the address function (ADR) or an address contact can be used. Access to the axis happens in the same way for all other function blocks.

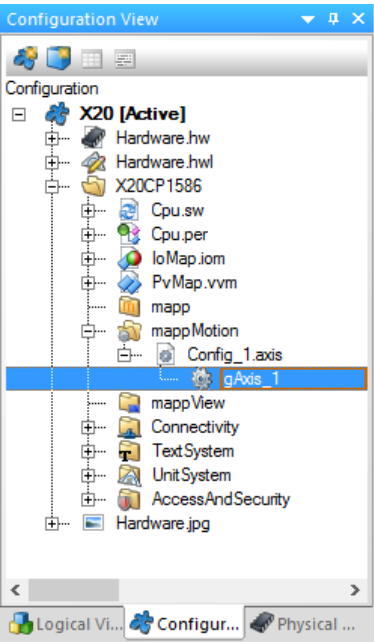


Figure 9: Axis reference in the Configuration View

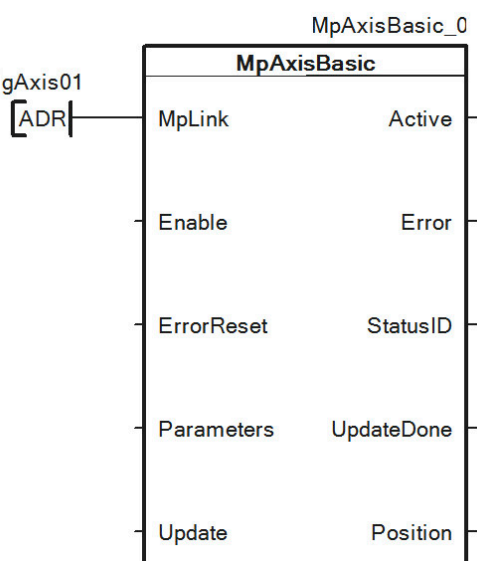


Figure 10: The axis reference is specified for the MpAxisBasic function block via the Address contact.

Specifying the motion parameters

For function block MpAxisBasic, it is necessary to transfer a data structure of type MpAxisBasicParType with the movement parameters. The data structure is preinstalled with the standard values. The following movement parameters can be transferred:

- Speed and acceleration
- Distance, position and direction of rotation
- Homing parameters
- Jog parameter

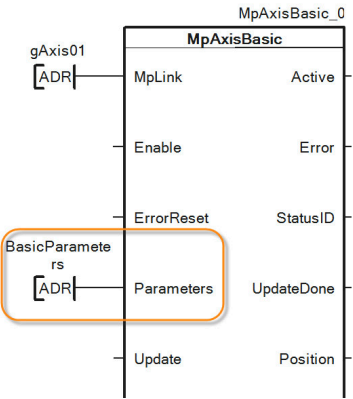


Figure 11: Transfer of the data structure with the movement parameters (MpAxisBasicParType)



Make sure that the initialization value "0" is not entered when setting up the data structures in the variable declaration. This is to retain the default value of the data structure.

| Name                         | Type               | & Reference              | Constant                 | Retain                   | Replicable                          | Value | Description [1] |
|------------------------------|--------------------|--------------------------|--------------------------|--------------------------|-------------------------------------|-------|-----------------|
| Function blocks              |                    |                          |                          |                          |                                     |       |                 |
| MpAxisBasic_0                | MpAxisBasic        | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input checked="" type="checkbox"/> |       |                 |
| Configuration and parameters |                    |                          |                          |                          |                                     |       |                 |
| BasicParameters              | MpAxisBasicParType | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input checked="" type="checkbox"/> |       |                 |

Figure 12: To initialize with default values, do not use an initialization value in the variable declaration.

Exercise: New program – Assign the axis reference and switch on the controller

A new program is created using the "Getting started" tutorial. The "Enable" input must be set to TRUE to enable MpAxisBasic. It is necessary to switch on the drive in order to prepare it<sup>2</sup>. If output "Active" = TRUE, then the controller is switched on via the "Power" input of the MpAxisBasic function block.

2 With the ACOPOSmulti system, power supply modules must also be switched on via the software.

- 1) Add the new program "m\_control".
- 2) Perform steps with the "Getting started" tutorial.



Motion control \ mapp Motion \ Getting started \ Axis

- 3) Transfer the program to the controller.
- 4) Set the input "Enable" to TRUE.
- 5) Wait until the outputs "Active" and "Info.ReadyToPowerOn" are TRUE.
- 6) Switch on the drive via "Power".
- 7) Monitor the status outputs of MpAxisBasic.



The "AxisParameters" structure is based on the MpAxisBasicParType data type. The structure is preinitialized with default values. It is important to ensure that the configured values for speed and acceleration match the configured system of units.



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Technology \ MpAxis \ Function blocks \ MpAxisBasic

### 3.3 Function block operation and status evaluation

This section briefly points out the operation of PLCopen-compliant function blocks. We will look at how to operate the function blocks as well as the options available for monitoring their operation. All function blocks are accessed using uniform operating parameters and return uniform status information. This simplifies the application and provides a better overview during programming.

#### Timing diagrams

In Automation Help, how the input states of the function blocks operate is illustrated with the aid of timing diagrams.

The example shows that the "Power" function is only available if the function block is active. It is necessary to set the "Enable" input to enable it. If input "Enable" is disabled when a controller is switched on and a movement is active, then no data is transferred to the axis. The axis continues to execute the last active commands. If an absolute movement is active and the MpAxisBasic function block is disabled, then the movement is continued.

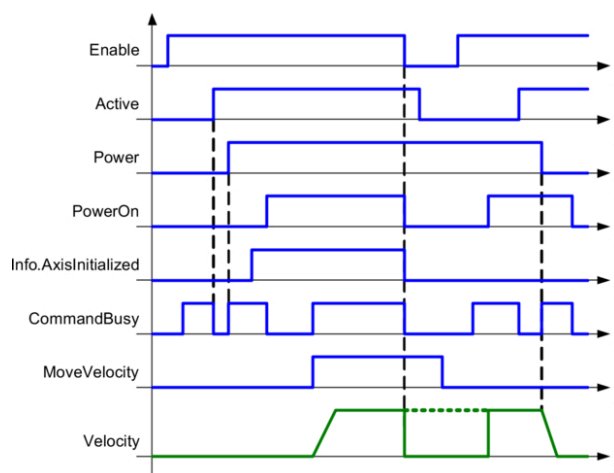


Figure 13: Timing diagram: Effect of input "Enable" on other commands and status outputs



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Technology \ MpAxis \ \ Function blocks \ MpAxisBasic \

#### Status information

In the event of an error, the value of output "Error" becomes TRUE. Output "StatusID" contains numerical information that can be searched for in Automation Help. Additional information about the current status is provided in output structure "Info".

If the cause of the error has been corrected and the error has been acknowledged using input "ErrorReset", then the controller can be switched on again with a positive edge on input "Power".

#### Exercise: Perform base movements using the Watch window

Base movements should be performed using the MpAxisBasic function block. In the first step, the necessary function block was already configured and called.

Command variables of the data type BOOL are now declared and connected to the inputs of MpAxisBasic. The individual commands are executed first via the Watch window.

##### The following commands should be used:

- "cmdPower" to switch on the controller
- "cmdHome" for homing the axis <sup>3</sup>
- "cmdErrorReset" to reset errors
- "cmdMoveVelocity" to move the axis using the speed setpoint
- "cmdMoveAdditive" to perform additive movements
- "cmdStop" to stop active movements

- 1) Declare command variables.
- 2) Connect command variables with the MpAxisBasic inputs.
- 3) Preconfigure the speed (10 units/second) and distance (100 units) using base parameters.
- 4) Switch on the controller ("cmdPower") and wait for the output "PowerOn".
- 5) Home the axis ("cmdHome") and wait for the output "isHomed".
- 6) Perform movements ("cmdMoveVelocity") and observe the output "InVelocity".
- 7) Check the status outputs and status structure of MpAxisBasic.

## 3.4 Overview of drive states

Defined states are determined in the PLCopen standard for operating a drive. They serve as an overview of the state in the positioning application as well as in the processing of error situations.

| State               | Description                                                                                       |
|---------------------|---------------------------------------------------------------------------------------------------|
| Disabled            | The drive controller is switched off.                                                             |
| Standstill          | The drive is not currently executing a movement and is ready for positioning commands.            |
| Homing              | The drive is executing a homing procedure.                                                        |
| ErrorStop           | The drive is at a standstill after an error. (Error stop).                                        |
| Stopping            | The drive is stopping an active movement.                                                         |
| Discrete Motion     | The drive is executing a movement to a target position. The movement therefore has a defined end. |
| Continuous Motion   | The drive is executing a movement without a target position. The movement has no defined end.     |
| Synchronized Motion | The drive is coupled to another drive.                                                            |

Table 2: Overview of the states in PLCopen Motion Control state diagram

<sup>3</sup> Homing mode "mcHOMING\_DIRECT" is used for this exercise.

The most important states of a positioning application are included in the PLCopen state diagram. These can be used for coordinating positioning sequences. The current PLCopen state of an axis can be read using the info structure of function block MpAxisBasic or using function block MC\_ReadStatus from the McAxis library.

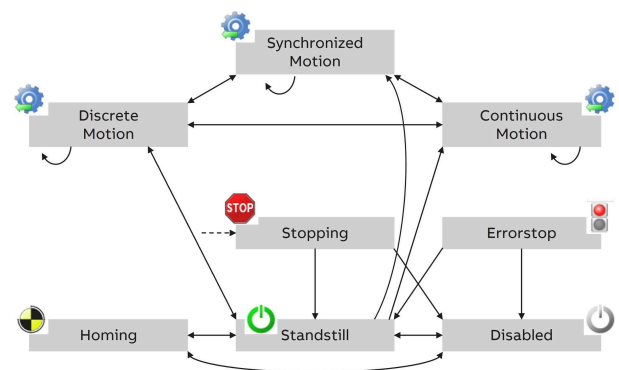


Figure 14: PLCopen Motion Control state diagram



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAxis \ Technical information \ State diagram



The transitions between these states can be initiated by calling individual drive functions. Let's assume that the axis is in the Standstill state. As soon as it has successfully performed a homing procedure, the "MoveAbsolute" command can be used to initiate a movement. After the target position has been reached, the drive returns to its initial state. If a drive error occurs during positioning, then the axis enters the error state ("Error" input). This can be acknowledged using the "ErrorReset" input once the drive error has been corrected.

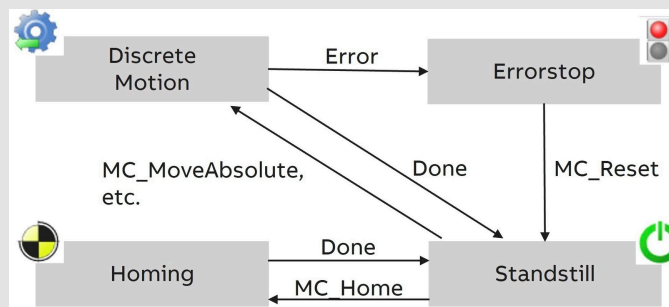


Figure 15: Sequence of states for performing an absolute movement

### Exercise: Read the PLCopen state

The current PLCopen state can be determined using the info structure of MpAxisBasic. The info structure can be displayed in the Watch window.

The controller is switched on and homing is performed. The current PLCopen status must be checked after each action.

- 1) Transfer the project.
- 2) Check the info structure in the Watch window on the output of MpAxisBasic.

### Exercise: Force a drive error, clear a drive error

The objective of this exercise is to trigger an error and check the Logger and status ID. The error is then reset via the "cmdErrorReset" input and the status outputs are monitored.

A base movement can be started when the controller is switched off, the maximum contouring error can be exceeded or the function block MC\_BR\_CommandError (library McAxis) can be used, for example.

- 1) Trigger a drive error.
- 2) Check the "Error" output and the Logger.

- 3) Acknowledge the errors via the "ErrorReset" input.
- 4) Check the "Error" output and the Logger.



Function block MC\_ReadAxisError reads the number (ID) of the currently pending axis error.

#### Exercise: Force errors on purpose

Function block MC\_BR\_CommandError can be used to trigger errors on purpose. The previous tasks should be combined with this function block.



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAxis \ Function blocks \ MC\_BR\_CommandError

#### Exercise: Automatic start

The existing program should now be somewhat automated. The MpAxisBasic function block should be enabled immediately after switching on the controller. In addition, the controller should be automatically switched on after successful enabling. Homing will then be carried out immediately. Additional commands are only permitted to be issued to the axis function block in this state. In the event of an error, all commands should be reset so that the controller can be switched on again once the acknowledgment is complete and so that all commands for starting movements are disabled.

- 1) Set up the state machine and configure the necessary steps.<sup>4</sup>
- 2) Set "cmdPower" if the "Info.ReadyToPowerOn" output = TRUE.
- 3) Set "cmdHome" if the "PowerOn" output = TRUE.
- 4) Reset "cmdHome" if "IsHomed" output = TRUE.
- 5) Reset all commands if the "Error" output = TRUE.

The drive is now prepared for performing base movements. The program can optionally be extended so that the closed-loop controller is automatically switched on and homed again after "cmdErrorReset" has been successfully executed.

<sup>4</sup> Creating a state machine is recommended when using high-level programming languages. When using visual programming languages such as Ladder Diagram, MpAxisBasic status information can be used for approving commands.

## 4 Programming tips

The application program is a way to represent an automatic sequence for controlling the drive. In this process, it is important to call the function blocks in the program in a clear manner. Furthermore, error events must be taken into consideration and responded to.

For a structured flow in the user program, a state machine can be used in high-level programming languages and jumps can be used in visual programming languages.

### 4.1 Uses of control structures

The function block offers inputs for controlling drive functions. Processes can be started at a defined point in the program using command variables. Status outputs and output parameters provide information about the current state of the respective function.

**The following information is provided by status outputs and output parameters:**

- Was the execution of the command successful?
- If not, what error occurred?
- Status of the process:
  - Is the axis moving?
  - Did the axis reach the target position?
  - Was homing performed successfully?

This information can be used for controlling the program sequence in the drive application. The program will have to respond differently depending on whether or not an error occurs.

The control structure that is especially suited for managing these types of function processes is the step switching mechanism (state machine)<sup>5,6</sup>

This type of structure allows the implementation of individual steps whose sequence can be determined by the use of a step index.

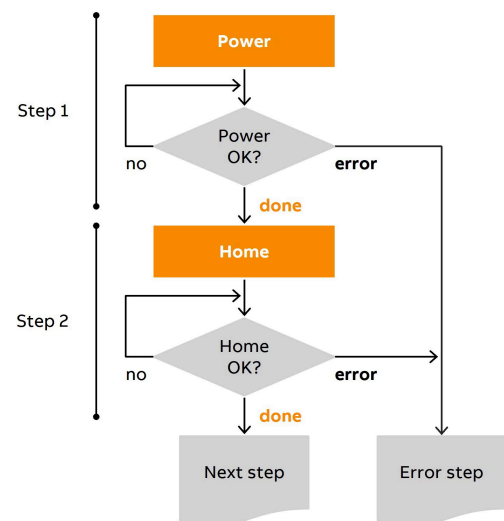


Figure 16: Sample control structure, structured programming

The necessary commands (switching on the controller, homing the axis, etc.) can be performed in the individual steps of the control structure. Status parameters such as "Error", "StatusID" and the info structure can be used to determine the step where the application will continue.

This gives the application a clear structure and opens it up for future expansion.

<sup>5</sup> All high-level programming languages have control structures that enable the implementation of a state machine or a step switching mechanism.

<sup>6</sup> In visual programming languages, functions are enabled via logical links of several digital signals.

## 5 Error indicators in the application program

There are basically several interfaces for troubleshooting mapp components (see 2.2 "Diagnostic options for mapp Technology components" on page 6).

Error handling can be programmed into the drive application using the "Error", "StatusID" and "Info" outputs. Error number descriptions can be found in Automation Help.

### Logger

Detailed data about the cause of the error is automatically entered in the "Motion" Logger file. The Logger entries can be opened using Automation Studio and System Diagnostics Manager and saved on the PC. Alternatively, the Logger file can be read using the AsEventLog library.

In mapp Cockpit, the last 10 Logger entries relevant for the respective view are displayed in the message area.



Diagnostics and service \ Diagnostics tools \ Logger  
 Diagnostics and service \ mapp Cockpit  
 Programming \ Libraries \ Configuration, system information, runtime control

- AsArLog
- ArEventLog

### Exercise: Reset command variables in the event of error

In the event of an error, all command variables of the application program must be reset automatically. When an error occurs, this is indicated via the "Error" and "StatusID" outputs.

- 1) Evaluate the "Error" output for positive edges.
- 2) Automatically reset the commands "cmdPower", "cmdHome" and "cmdMoveAdditive" in the event of an error.

### Exercise: Read the axis error with the Logger, acknowledge the axis error

If an attempt is made to start a movement without turning on the controller, for example, an error state is triggered. The error state is indicated via the MpAxisBasic function block outputs. A detailed description of the cause of the error is logged in the "Motion" Logger file. After diagnostics have been completed, the error should be acknowledged via the "ErrorReset" input.

- 1) Start the movement with a switched-off controller.
- 2) Open the Logger.
- 3) Search for the error number description in Automation Help<sup>7</sup>.
- 4) Analyze the cause of error and acknowledge the drive error.



Function block input "ErrorReset" resets all errors and puts the axis into state "Disabled". After the acknowledgment is completed, it is possible to switch on the controller on a positive edge on the "Power" input again.

### Exercise: Switch on the controller automatically after successful ErrorReset

After all errors have been successfully reset, the controller should be automatically switched on again via the drive application.

<sup>7</sup> The help page with the error description is opened by marking the error entry and pressing the <F1 key> in the Logger.

- 1) Evaluate the "Error" output for negative edges.
- 2) Switch on the controller again with the "cmdPower" command.
- 3) Perform a homing procedure, if required.

## 6 Additional MpAxisBasic functions

The MpAxisBasic function block offers more functions in addition to those used for drive preparation and for performing base movements. In addition to the functions already in use, torque limitation and jog mode are also possible.



The optional parameters of function blocks can be displayed in Automation Help by enabling the "Optional parameters" checkbox above the image of the function block instance.



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Technology \ MpAxis \ Function blocks \ MpAxisBasic \ Description

### Exercise: Implement torque limiting

Specifying torque limiting in drive control takes place directly via the MpAxisBasic parameter structure. The torque should be limited to 0.5 Nm, for example.

- 1) Specify the torque limit value.
- 2) Switch on the drive and perform a homing procedure.
- 3) Start the movement.
- 4) Enable the torque by enabling the "LimitLoad" input.
- 5) Check the behavior of the actuator controls during torque limitation.

# 7 The PLCopen Motion library (McAxis)

The PLCopen Motion library is called library **McAxis** in Automation Studio. It contains standardized PLCopen function blocks to control B&R drive solutions.

Basic functions such as homing or base movements are defined in the library McAxis. This provides the user with a uniform software interface for a specific area of standard applications.

The actual range of B&R drive solution functions goes far beyond that which is contained in the standard.

For example, expanded basic positioning and powerful function blocks for coupling axes are available as a complement to the standard functions<sup>8</sup>.



Figure 17: PLCopen Motion Control logo

The McAxis library has been expanded to include additional B&R-specific functions that allow the user to take advantage of these functionalities in a uniform manner. These advanced functions are operated in compliance with the PLCopen standard.

In addition, device-specific functions are available in other libraries (e.g. for ACOPOS servo family: McAcPax).



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAxis  
Drive technology \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAcPax

## 7.1 Compatibility of McAxis and MpAxis

The function blocks in library MpAxis (e.g. MpAxisBasic) are based on the function blocks in library McAxis. The two libraries are completely compatible with each other. In both libraries, the axis is accessed via the same axis reference. Parameters and commands for the drive are transferred when commands are triggered. The state in the PLCopen state diagram then changes. This structure makes it possible to add features to applications that have been programmed with MpAxisBasic using function blocks from library McAxis.

The function compatibility, for example, enables the controller to be switched on and homed with MpAxisBasic and to start a relative movement on the same axis with function block MC\_MoveAdditive from library McAxis. The change of state from "Standstill" to "Discrete Motion" is shown immediately in the info structure of the MpAxisBasic function block.



B&R tutorials:

[Course \ mapp Technology \ mapp Motion \ TM416 – Programming mapp Axis \ MpAxis and McAxis combination](#)



### Exercise: Combine mapp Technology and library McAxis

The MpAxisBaisc function block is based on the McAxis function blocks. For this reason, it is compatible with the function blocks from other mapp Motion libraries (like any other mapp Motion function block).

The MpAxisBasic function block offers all relevant functions of an axis component, but it is possible to expand the motion application with function blocks from the McAxis library. The drive components are accessed in both cases via the axis reference.

In this task, the drive should be prepared using the MpAxisBasic function block and a movement should be performed using the MC\_MoveVelocity function block from library McAxis.

- 1) Add the MC\_MoveVelocity function block to the application.
- 2) Switch on the controller and perform a homing procedure using MpAxisBasic.

<sup>8</sup> A detailed overview of multi-axis and coupling functions is provided in the training module "TM417 – Axis coupling, mapp Axis".

- 3) Start a movement in the positive direction of rotation with MC\_MoveVelocity.
- 4) Check the outputs of MpAxisBasic for a status change.



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAxis \ Function blocks \ MC\_MoveVelocity

### Exercise: Specify the cyclic position

The goal of this exercise is to combine MpAxis functions with those from PLCopen-compliant libraries, e.g. McAxis. Function block MC\_BR\_MoveCyclicPosition is used to extend the range of functions so that a setpoint position is specified cyclically for the axis.

#### The following program logic must be used:

- If the input "moveforward" is true, the axis should move clockwise by 0.1° with each task class call (every 2 ms).
- If the input "movebackward" is true, the axis should move counterclockwise by 0.1° with each task class call (every 2 ms).
- If a movement with MpAxisBasic was executed before MC\_BR\_MoveCyclicPosition, then the first call of input "MC\_BR\_MoveCyclicPosition.CyclicPosition" must correspond to the last position of output "MpAxisBasic.Position". Otherwise, the first cyclically specified position is 0.0.



Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAxis \ Function blocks \ MC\_BR\_MoveCyclicPosition

Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAcpAx \ Function blocks \ Supported function blocks from library McAxis \ MC\_BR\_MoveCyclicPosition

Motion control \ mapp Motion \ Programming \ Application program \ Libraries \ Core \ McAcpAx \ Technical information \ Broadcast with POWERLINK

- 1) Configure the POWERLINK broadcast channel.
- 2) Add the McAxis library and function block MC\_BR\_MoveCyclicPosition to the project.
- 3) Cyclically specify the axis reference at the "Axis" input with ADR() (as with "MpLink").
- 4) Create the cyclic programming logic.
- 5) Transfer the program.
- 6) Change the variables "moveforward" and "movebackward" in the Watch window.
- 7) Record trace in mapp Cockpit.



Here, "movebackward" became TRUE after 3 seconds; "moveforward" became TRUE after 8 seconds. Using the cursors, you can see that the axis has rotated 50° counterclockwise within one second.

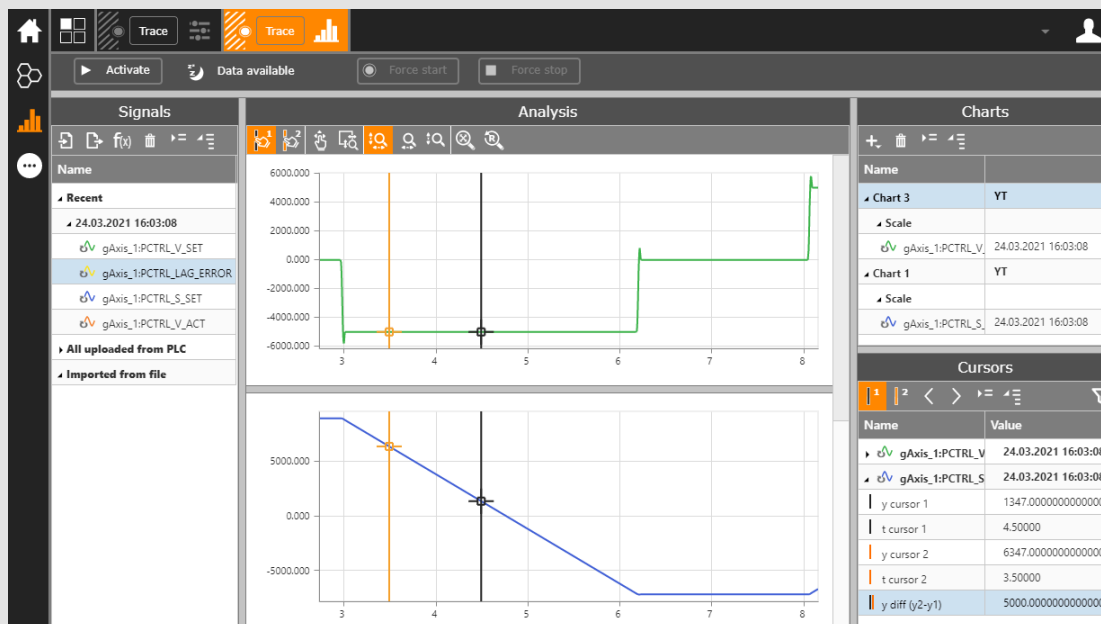


Figure 18: Green = Speed, Blue = Distance

### Optional exercise: Switch between a position movement and torque movement

The goal of this exercise is to get to know how seamlessly function blocks from different libraries can intervene with one another.

- 1) Start a movement with function block MpAxisBasic (e.g. MoveVelocity).
- 2) Start a movement using MC\_TorqueControl and abort the previous movement.
- 3) Analyze the function block outputs.
- 4) Holding the axis mechanically in the MpAxisBasic case controls the movement and MC\_TorqueControl. What is the difference here?

## 8 Automation

With mapp Motion, B&R provides a uniform approach to motion control programming. This solution includes components for controlling single-axis movements as well as CNC and robotics applications. As part of the mapp software framework, other mapp components can be combined as needed and easily configured. Users benefit from fast and easy project development.

With B&R's mapp View HMI solution, you can give your machine tool a state-of-the-art web-based user interface even without special web design training. B&R Automation Studio software development is used for configuration.



mapp Technology \

### 8.1 Automating an axis application

#### Exercise: Simple level of automation

The goal of this exercise is to easily manage the program.

The following requirements must be implemented; the existing procedure (error handling) is expanded:

- Switching on via a variable should enable the controller.
- Home the axis if necessary.
- Perform a movement to a starting position (if not already there).
- An additional command triggers a movement sequence (relative movement → torque movement → absolute movement)

#### Optional exercise: Associated movement

The goal of this exercise is to implement and understand an example that summarizes the contents of this training module and the training itself over the past few days.

Requirements for moving the axis:

- Any real axis should be used, periodic system related to a rotation of the motor, no gear ratio.
- Axis preparation when switching on controller, homing should take place automatically.
- Starting point is position 0.
- The procedure should run one time automatically when setting variable "cmdStart".
- The axis should move 3 motor rotations to the left.
- The axis should then remain at a standstill for exactly 600 ms.
- Next, the axis should move 7 motor rotations to the right.
- It should then wait 1.5 seconds at this position.
- A concluding absolute movement to position 0 should then take place.
- During the entire sequence, record a trace (with trigger setting, do not record longer than necessary!) with maximum resolution.

It should include position, speed and useful information about the drive and controller.



A time block (e.g. TON) or counter variable (e.g. cntCycles) can be used for this exercise to implement the required time delay.

#### Optional exercise: Clock replication

The objective of this exercise is to display the seconds and minutes of a clock on 2 motors within one revolution each. The exercise can be divided into 3 objectives:

- Determining the time
- Converting minutes and seconds to absolute positions for the motors
- Sending the command for an absolute movement to the axes

The exercise can be solved in several ways, and here are a few hints for a solution:

- Library "astime" can be used to determine the current time (DTGetTime()) and DT\_TO\_DTStructure())
- The axes should be defined as periodic axes with exactly one revolution (for absolute movement in the positive direction)
- The task class for controlling MpAxisBasic should have a 500 ms cycle time. This allows the absolute movement to be started in one cycle and the input to be reset in the next cycle in order to start again.

## 9 Summary



Figure 19: A multitude of possibilities for programming with mapp Motion

### Single-axis positioning made easy

With mapp Axis, B&R provides a fast and easy way to program single-axis positioning tasks. mapp Axis is practical in focus and comes with preconfigured blocks. This makes Programming considerably faster and easier.

Axes are programmed in the Automation Studio development environment using either PLCopen Part 1 function blocks or mapp components. mapp Axis provides a complete package of preconfigured modules for typical single-axis applications, such as integrated alarm handling and easy axis coupling. This makes the handling of the drive system considerably easier.

### Faster project implementation

The provided mapp components allow for quicker and easier project implementation. The programmed applications can also be reused and are easier to maintain. At B&R, software and hardware are completely decoupled, so the hardware in a project can easily be switched out at any time.

Configuration is practically oriented. Rather than specific drive settings, all the developer needs to know is where the axis will be used and what it will be used for. Individual axes can be configured as linear or rotary axes, for example, or the developer can set the physical units in which the axis should do its positioning. This further simplifies project development.

### Uniform axis management

To simplify the process of creating applications even further, B&R's system allows axes to be grouped into administrative axis groups. This is helpful in cases where axes have a common or uniform use, including startup, homing or error handling functions. In the application program, axis management occurs independently of the number and type of axes used.

### POWERLINK DS402 drives also supported

mapp Axis supports all B&R drives and DS402-compatible third-party drives with a POWERLINK interface. Developers can program DS402 drives in the same ways as B&R drives – using mapp Technology or PLCopen function blocks. The drives are integrated into the automation project just like any B&R drive. For this to be possible, the DS402 drives must adhere to the applicable CAN in Automation standard as defined in IEC 61800-7-201.

**Highlights**

- Ready-made components accelerate programming
- Multiple axes can be managed in a group
- Practically-oriented axis configuration
- Support for all B&R and DS402 drives
- Available for many B&R stepper motor modules

The enhancement of the drive application is possible because of function compatibility with the other mapp Motion libraries, for example with the function blocks in libraries McAxis, McAcpAxis, etc.

# 10 Solutions

## 10.1 Solution: Switch on the controller automatically and perform direct homing

```

PROGRAM _INIT

    (*set basic parameters*)
    BasicParameters.Velocity := 1000; (*1000 units/second*)
    BasicParameters.Distance := 1000; (*1000 units distance*)

END_PROGRAM

PROGRAM _CYCLIC

    CASE sStep OF
        enINIT:
            (*do nothing; sequence is started manually*)

        enSTART:
            MpAxisBasic_0.Enable := 1;

            (*is axis ready to be powered on?*)
            IF (MpAxisBasic_0.Active = TRUE) AND
                (MpAxisBasic_0.Info.ReadyToPowerOn = TRUE) THEN
                sStep := enPOWER_ON;
            END_IF

        enPOWER_ON:
            cmdPower := TRUE;

            (*axis powered on?*)
            IF MpAxisBasic_0.PowerOn = TRUE THEN
                sStep := enHOME;
            END_IF

        enHOME:
            cmdHome := TRUE;

            (*axis homed?*)
            IF MpAxisBasic_0.IsHomed = TRUE THEN
                sStep := enOPERATION;
                cmdHome := FALSE;
            END_IF

        enOPERATION:
            (*commands for basic movements*)
            cmdMoveVelocity;
            cmdMoveAdditive;
            cmdStop;

            (*reset cmdMoveAdditive command when position reached*)
            IF EDGEPOS (MpAxisBasic_0.InPosition) = TRUE THEN
                cmdMoveAdditive := FALSE;
            END_IF

            (*update parameters*)
            cmdUpdate;

            (*axis in error step?*)
            IF MpAxisBasic_0.Error = TRUE THEN

```

```

        sStep := enERROR;
    END_IF

enERROR:
    (*power off*)
    cmdPower := FALSE;

    (*reset all commands*)
    cmdMoveVelocity := FALSE;
    cmdMoveAdditive := FALSE;
    cmdStop := FALSE;
    cmdUpdate := FALSE;

    (*implement error handling here*)
    cmdErrorReset;
    IF MpAxisBasic_0.Error = FALSE THEN;
        sStep := enSTART;
        cmdErrorReset := FALSE;
    END_IF

END_CASE

MpAxisBasic_0.MpLink := ADR(gAxis01);
MpAxisBasic_0.Parameters := ADR(BasicParameters);

(*execute commands*)
MpAxisBasic_0.Power := cmdPower;
MpAxisBasic_0.Home := cmdHome;
MpAxisBasic_0.ErrorReset := cmdErrorReset;
MpAxisBasic_0.MoveVelocity := cmdMoveVelocity;
MpAxisBasic_0.MoveAdditive := cmdMoveAdditive;
MpAxisBasic_0.Stop := cmdStop;
MpAxisBasic_0.Update := cmdUpdate;

(*call all mapp components*)
MpAxisBasic_0();

(*movement with MC_MoveVelocity to watch status
changes OF the MpAxisBasic component*)
MC_MoveVelocity_0.Axis := ADR(gAxis01);
MC_MoveVelocity_0.Execute := cmdMoveVelocity_ACP10_MC;
MC_MoveVelocity_0.Velocity := 1000;
MC_MoveVelocity_0.Acceleration := 2000;
MC_MoveVelocity_0.Deceleration := 2000;
MC_MoveVelocity_0.Direction := mcPOSITIVE_DIR;

(*call all ACP10_MC function blocks*)
MC_MoveVelocity_0();

END_PROGRAM

PROGRAM _EXIT

    (* Choose "Keep PV values" for the download behavior.
    It is recommended when using mapp Motion or ACP10/ARNCO *)

END_PROGRAM

```

### Solution for clock replication

Subsequently, the portion of the exercise that determines how the motor position can be calculated based on the current second or minute is programmed.

MpAxisBasic can be operated cyclically in a 500 ms task class. The positions must be copied to the MpAxisBasic parameter structure and MoveAbsolute moves the axis to the positions.

Use one axis for seconds and the other axis for minutes.

Variable declaration:

```
VAR
    DTGetTime_0 : DTGetTime;
    actStructDateTime : DTStructure;
    MeasurementUnitsPerRevolution : LREAL;
    AbsolutPosMin : LREAL;
    AbsolutPosSec : LREAL;
END_VAR
```

Possible program code:

```

PROGRAM _INIT
    DTGetTime_0.enable := TRUE;
    MeasurementUnitsPerRevolution := 1.0;
END_PROGRAM

PROGRAM _CYCLIC
    DTGetTime_0();    (* Get actual time *)
    (* Convert to structure for easy access *)
    DT_TO_DTStructure(DTGetTime_0.DT1, ADR(actStructDateTime));

    (* Calculate absolute position for periodic axes *)
    AbsolutPosMin := MeasurementUnitsPerRevolution / 60 *
        USINT_TO_LREAL(actStructDateTime.minute);
    AbsolutPosSec := MeasurementUnitsPerRevolution / 60 *
        USINT_TO_LREAL(actStructDateTime.second);
END_PROGRAM

```

Possible control of the absolute movement in a 500 ms task class:

```

(* After axes are prepared, this can be done in e.g. 500ms cycle time *)
IF (cmdMove) THEN
    IF (cmdStart) THEN
        cmdStart := FALSE;
    ELSE
        cmdStart := TRUE;
    END_IF
END_IF

AxisParameters_Min.Position := AbsolutPosMin;
MpAxisBasic_Min.MoveAbsolute := cmdStart;
MpAxisBasic_Min();    (* Call FB *)

AxisParameters_Sec.Position := AbsolutPosSec;
MpAxisBasic_Sec.MoveAbsolute := cmdStart;
MpAxisBasic_Sec();    (* Call FB *)

```

# Automation Academy

## Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom Learning



Virtual Classroom  
Learning



Online courses

## Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of training course content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

## Virtual classroom

**Remote Lectures** supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

## Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a targeted choice of information to be used as a reference.

## Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



**AutomationAcademy**





**B&R**  
**Industrial Automation GmbH**  
**A member of the ABB Group**  
B&R Straße 1  
5142 Eggelsberg, Austria  
office@br-automation.com

t +43 7748 6586-0  
f +43 7748 6586-26

**br-automation.com**



TM416TRE.492-ENG

V3.0.0.1 ©2023/09/28 by B&R, All rights reserved.  
All registered trademarks are the property of their respective owners.  
Subject to technical changes without notice.

